Phase 2 | Ready Scout Implementation Deep-Dive Series

An index to the nine implementation-focused volumes.

React/Vite Edition | Repository-grounded study material | Snapshot: 2026-10-24

How to use the deep dives

These volumes complement the nine interview-preparation phases. Read them with the source repository open. Each volume follows the current Ready Scout execution path and separates implemented behavior from recommendations. The package is independent of the MetroInsights reference archive.

Volumes

Volume	Focus and key source files
01 Routes and request flow	React Router, Vite proxy, Express startup, mounted routes, deployment rewrites.
02 Authentication and user data	Mongoose User, bcrypt, JWT, Passport Google OAuth, profile and refresh routes.
03 Alerts and CAP processing	SACHET summary route, CAP XML parser, ticker, detail page, USGS/GDACS utilities.
04 Nearby search and map	Geolocation, Geoapify request, cache/coalescing, normalization, Haversine distance, Leaflet.
05 Chatbot and Gemini flow	Chat UI, Flask /chat, system instruction, Gemini request and response parsing.
06 Earthquake game and learning	Phaser setup/scenes/controls/collisions/results, quiz project boundary.
07 Frontend architecture and deployment	Vite, React Router, AuthContext, lazy map, Vercel/Render and local configuration.
08 End-to-end execution trace	Boot, sign-in, dashboard, alert, nearby lookup, game, chat and failure diagnosis.

Repository map for the series

- frontend/src/main.jsx and App.jsx: web bootstrap and routes.
- frontend/src/app/: dashboard, auth, context and reusable UI components.
- backend/index.js: middleware, service setup, API route mounts and static fallback.
- backend/routes/: auth, user, alerts, nearby and uploads.
- backend/models/User.js: MongoDB schema and password/JWT methods.
- flask/app.py: Gemini-backed chat microservice.
- frontend/public/earthquake_drill/: Phaser game.
- earthquake-quiz/: separate Vite quiz application.

Cross-cutting study questions

- What executes in the browser, Express, MongoDB and Flask?
- What environment variables configure each service?
- Which inputs cross a trust boundary and how are they validated?
- Which upstream provider can fail, and how is that failure surfaced?
- Which behaviors are present today versus recommended next steps?

Snapshot note: the nearby route currently calls Geoapify Places and requires a backend key. The project contains other map data utilities; verify whether each is wired to a visible flow before claiming it is active.